FaceMotion

Facial Therapy Application

TECHNICAL IMPLEMENTATION

1. Technology Stack

Technology	Use in the project
Unreal Engine 5.7	Desktop runtime, real-time character rendering, animation, audio and application execution
MetaHuman	Therapist and patient character assets and facial animation
C++	Backend communication, WebUI bridge, application state, curve logging, comparison and session handling
Blueprints	Runtime configuration, asset assignment, application events and project-level control
HTML, CSS and JavaScript	Embedded user interface, responsive layouts and German/English presentation
Unreal WebBrowser and UMG	Hosts the WebUI inside the Unreal application
HTTP and JSON	REST API requests, backend responses and generated WebUI data

Interface Implementation

The application interface is built as local HTML, CSS and JavaScript pages stored under `Content/WebUI`. Unreal displays these pages through the WebBrowser widget. This replaces the need to build each screen as a native Unreal UMG widget.

UMG remains the Unreal container for the browser and runtime integration. The visual layout, page navigation and most text rendering are handled by the WebUI.

Runtime Layers

1	2	3	4
WebUI	**C++ Bridge**	**Blueprint / Unreal**	**Backend API**
HTML pages, controls and presentation	Communication and state conversion	Characters, assets and runtime events	Persistent account and session data

2. Data Exchange

WebUI to Unreal

A C++ bridge object is bound to the WebBrowser as `UEInterface`. JavaScript calls the exposed bridge functions for actions such as login, exercise start, pause, resume, language selection and camera events.

> **JavaScript → UEInterface → C++ → Blueprint / Exercise System**

Unreal to WebUI

C++ sends live results back by executing JavaScript in the browser. Named JavaScript callbacks receive login results, exercise-start status, roadmap status and live exercise progress.

> **C++ → ExecuteJavascript → JavaScript callback → Page update**

Page Data

Backend responses are converted into page-specific JSON objects. C++ writes both a JSON file and a JavaScript data file containing `window.dashboardData`. The page loads the generated JavaScript file and renders the current data.

1. **C++ requests data** from the backend using Unreal's HTTP module.
2. **The JSON response is parsed** into Unreal structures.
3. **Page data is generated** for the required WebUI screen.
4. **The WebUI loads the generated data** and updates the visible page.

Backend Communication

API communication is centralized in the C++ API layer. Authentication tokens and the current user, patient, mode, avatar and language values are stored for reuse by C++ and Blueprints. Session and progress updates are submitted through the same layer.

3. Curve Logging and Comparison

Curve Capture

The comparison system reads animation curves from the therapist and patient skeletal mesh animation instances. Attribute, material and morph-target curves are captured into timestamped frames at a configurable sampling interval.

Live Comparison

1. **Therapist and patient curves are sampled** during the active comparison stage.
2. **Relevant facial curves are selected** and mirrored curve names are supported where required.
3. **Curve differences are evaluated** using configurable tolerance and curve weighting.
4. **Regional and individual-curve similarity are combined** into the current accuracy value.
5. **Accuracy is smoothed and sampled** throughout the comparison window.
6. **The final score is calculated** from the recorded accuracy samples.

Runtime Feedback

- The current accuracy, remaining time, progress and status are exposed to the WebUI.
- Low accuracy can temporarily pause therapist animation according to the configured threshold logic.
- A Blueprint event is broadcast when five seconds remain in an active comparison.
- The average score is used for feedback and completion validation.

Logging Output

Therapist and patient curve frames can be exported as JSON under the project `ExportedCurves` directory. Each frame contains its timestamp and captured curve values. This output is used for development and comparison diagnostics.

4. Exercise and Session Integration

Exercise Configuration

Facial exercise montages are loaded from an Unreal DataTable. Guidance configuration contains the audio and facial montage assigned to each stage. Separate English and German guidance configurations can be selected from the current language.

Exercise Runtime

C++ controls the exercise stages, montage playback, timers, comparison windows, rest periods, feedback and progress state. Blueprints supply meshes, DataTables, montages, audio assets and event handling without duplicating the comparison logic.

Session Persistence

Exercise completion is submitted to the backend from the curve-logging layer. The submitted data includes the session reference, mode, exercise identifiers and calculated accuracy. Progress is updated only when the completion requirements are met.

Error Handling

- Backend and exercise-start errors are retained in C++ and returned to the WebUI.
- Patient-specific page data is regenerated before rendering the relevant screen.
- Generated WebUI cache files can be cleared when returning to login.
- Blueprint-callable status and debug functions are available for runtime checks.

> **Implementation boundary:** WebUI handles presentation and user interaction. C++ handles backend access, state, logging and comparison. Blueprints connect Unreal assets and runtime events to the C++ systems.